

Algorithms for Programming Contests - Week 11

Prof. Dr. Javier Esparza,
Vincent Fischer,
Jakob Schulz,
conpra@model.cit.tum.de

13. Januar 2026

Trie

A *Trie* is a tree data structure that is used to store a set of strings:

Trie

A *Trie* is a tree data structure that is used to store a set of strings:

- Root node represents empty string.

Trie

A *Trie* is a tree data structure that is used to store a set of strings:

- Root node represents empty string.
- Outgoing edges are associated with a label (e.g. a letter).

Trie

A *Trie* is a tree data structure that is used to store a set of strings:

- Root node represents empty string.
- Outgoing edges are associated with a label (e.g. a letter).
- Path from root to a node represents a prefix of a word/words.

Trie

A *Trie* is a tree data structure that is used to store a set of strings:

- Root node represents empty string.
- Outgoing edges are associated with a label (e.g. a letter).
- Path from root to a node represents a prefix of a word/words.
- All descendants of a node have the same prefix.

Trie

A *Trie* is a tree data structure that is used to store a set of strings:

- Root node represents empty string.
- Outgoing edges are associated with a label (e.g. a letter).
- Path from root to a node represents a prefix of a word/words.
- All descendants of a node have the same prefix.
- Position of a node in the trie defines the associated string.

Trie

A *Trie* is a tree data structure that is used to store a set of strings:

- Root node represents empty string.
- Outgoing edges are associated with a label (e.g. a letter).
- Path from root to a node represents a prefix of a word/words.
- All descendants of a node have the same prefix.
- Position of a node in the trie defines the associated string.
- Nodes at which a word ends are tagged.

Trie

A *Trie* is a tree data structure that is used to store a set of strings:

- Root node represents empty string.
 - Outgoing edges are associated with a label (e.g. a letter).
 - Path from root to a node represents a prefix of a word/words.
 - All descendants of a node have the same prefix.
 - Position of a node in the trie defines the associated string.
 - Nodes at which a word ends are tagged.
-
- Invented by René de la Briandais in 1959.

Trie

A *Trie* is a tree data structure that is used to store a set of strings:

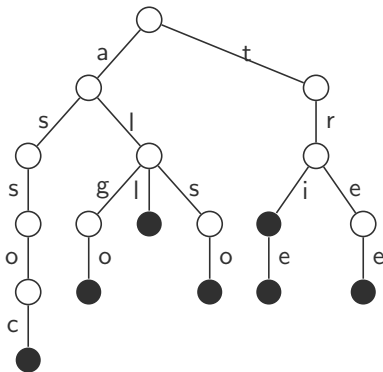
- Root node represents empty string.
 - Outgoing edges are associated with a label (e.g. a letter).
 - Path from root to a node represents a prefix of a word/words.
 - All descendants of a node have the same prefix.
 - Position of a node in the trie defines the associated string.
 - Nodes at which a word ends are tagged.
-
- Invented by René de la Briandais in 1959.
 - Name originates from the term *retrieval*.

Trie - Applications

- Storing a dynamic set of strings.
- Sorting strings lexicographically.
- Autocompletion
- Spell-checking

Trie

The next figure shows a trie with the words “tree”, “trie”, “tri”, “algo”, “assoc”, “all”, and “also”.



Trie - Implementation details

As an example, an array of the following struct can be used:

```
struct node{  
    int successors[26]; // Size of Alphabet  
    bool end;  
};
```

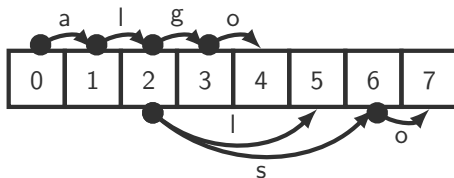
Or

```
int successors[N][26]; // size of Alphabet  
bool end[N];
```

Here, one can set `successors[i][j]` to 0 if node i has no successor w.r.t. j .

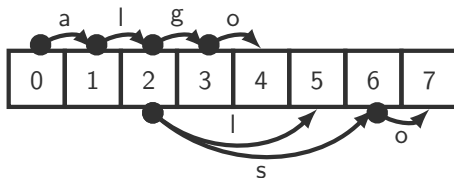
Trie - Implementation details

Here is an example of a storage structure for the words inserted in this order: "algo", "all", "also"



Trie - Implementation details

Here is an example of a storage structure for the words inserted in this order: “algo”, “all”, “also”



For large alphabets: replace successors[26] with HashMap
symbol $\mapsto$ *target*, or use HashMap (*source*, *symbol*) $\mapsto$ *target*

Trie - Operations

LOOKUP(S)

INSERT(S)

(Optional) DELETE(S)

Trie - Operations

LOOKUP(s)

- Start at the root node and traverse edges w.r.t. letters of s .

INSERT(s)

(Optional) DELETE(s)

Trie - Operations

LOOKUP(s)

- Start at the root node and traverse edges w.r.t. letters of s .
- If no suitable edge exist, s is not contained in the trie.

INSERT(s)

(Optional) DELETE(s)

Trie - Operations

LOOKUP(s)

- Start at the root node and traverse edges w.r.t. letters of s .
- If no suitable edge exist, s is not contained in the trie.
- If the whole word was processed, check whether the current node marks the end of a word.

INSERT(s)

(Optional) DELETE(s)

Trie - Operations

LOOKUP(s)

- Start at the root node and traverse edges w.r.t. letters of s .
- If no suitable edge exist, s is not contained in the trie.
- If the whole word was processed, check whether the current node marks the end of a word.

INSERT(s)

- Start at the root node and traverse edges w.r.t. letters of s .

(Optional) DELETE(s)

Trie - Operations

LOOKUP(s)

- Start at the root node and traverse edges w.r.t. letters of s .
- If no suitable edge exist, s is not contained in the trie.
- If the whole word was processed, check whether the current node marks the end of a word.

INSERT(s)

- Start at the root node and traverse edges w.r.t. letters of s .
- Add non-existing edges along the way.

(Optional) DELETE(s)

Trie - Operations

LOOKUP(s)

- Start at the root node and traverse edges w.r.t. letters of s .
- If no suitable edge exist, s is not contained in the trie.
- If the whole word was processed, check whether the current node marks the end of a word.

INSERT(s)

- Start at the root node and traverse edges w.r.t. letters of s .
- Add non-existing edges along the way.
- Tag last node as the end of a word.

(Optional) DELETE(s)

Trie - Operations

LOOKUP(s)

- Start at the root node and traverse edges w.r.t. letters of s .
- If no suitable edge exist, s is not contained in the trie.
- If the whole word was processed, check whether the current node marks the end of a word.

INSERT(s)

- Start at the root node and traverse edges w.r.t. letters of s .
- Add non-existing edges along the way.
- Tag last node as the end of a word.

(Optional) DELETE(s)

- Easiest: Just untag the end node corresponding to s

Trie - Operations

LOOKUP(s)

- Start at the root node and traverse edges w.r.t. letters of s .
- If no suitable edge exist, s is not contained in the trie.
- If the whole word was processed, check whether the current node marks the end of a word.

INSERT(s)

- Start at the root node and traverse edges w.r.t. letters of s .
- Add non-existing edges along the way.
- Tag last node as the end of a word.

(Optional) DELETE(s)

- Easiest: Just untag the end node corresponding to s
- However, as a consequence, there may be untagged subtrees $\rightarrow$ may need to handle (by deletion), in which case one should prefer a linked data structure over an array

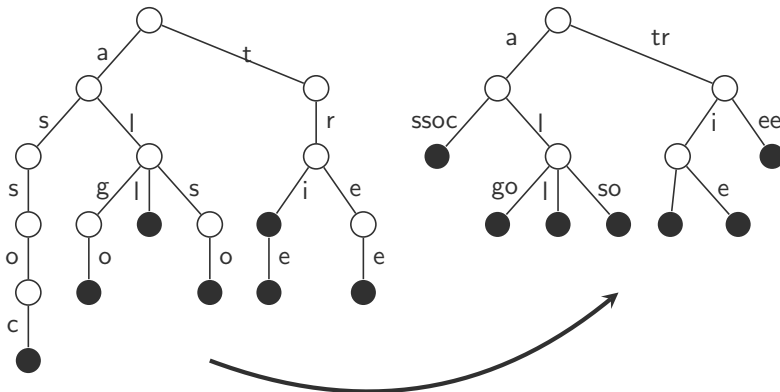
Trie - Complexity

Time complexity:

- INSERT(s): $\mathcal{O}(|s|)$
- LOOKUP(s): $\mathcal{O}(|s|)$
- DELETE(s): $\mathcal{O}(|s|)$ if using a linked list

Trie - Remarks

- For sparse string sets, the tree can be "compressed" by merging "only childs" ("Einzelkinder") with their parent. The resulting structure is called a **Radix tree**.



Segment Tree - Motivation

Given an integer array $a[n]$. Implement two operations:

- $\text{ADD}(i, v)$: Add value v to i -th element.
- $\text{SUM}(\ell, r)$: Sum up all elements in interval $[\ell, r]$.

Segment Tree - Motivation

Given an integer array $a[n]$. Implement two operations:

- $\text{ADD}(i, v)$: Add value v to i -th element.
- $\text{SUM}(\ell, r)$: Sum up all elements in interval $[\ell, r]$.

Alg 1:

- $\text{ADD}(i, v)$: Simply add v to $a[i]$.
- $\text{SUM}(\ell, r)$: Loop from $a[\ell]$ to $a[r]$ and sum up values.

Segment Tree - Motivation

Given an integer array $a[n]$. Implement two operations:

- $\text{ADD}(i, v)$: Add value v to i -th element.
- $\text{SUM}(\ell, r)$: Sum up all elements in interval $[\ell, r]$.

Alg 1:

- $\text{ADD}(i, v)$: Simply add v to $a[i]$.
- $\text{SUM}(\ell, r)$: Loop from $a[\ell]$ to $a[r]$ and sum up values.
- Complexity: $\text{ADD}(i, v) \in \mathcal{O}(1)$, $\text{SUM}(\ell, r) \in \mathcal{O}(n)$

Segment Tree - Motivation

Alg 2:

- Compute prefix sums and store them in $b[]$.
- $\text{ADD}(i, v)$: Add v to $a[i]$ and recompute $b[]$.
- $\text{SUM}(\ell, r)$: Return $b[r] - b[\ell - 1]$.

Segment Tree - Motivation

Alg 2:

- Compute prefix sums and store them in $b[]$.
- $\text{ADD}(i, v)$: Add v to $a[i]$ and recompute $b[]$.
- $\text{SUM}(\ell, r)$: Return $b[r] - b[\ell - 1]$.
- Complexity: $\text{ADD}(i, v) \in \mathcal{O}(n)$, $\text{SUM}(\ell, r) \in \mathcal{O}(1)$

Segment Tree - Motivation

Alg 2:

- Compute prefix sums and store them in $b[]$.
- $\text{ADD}(i, v)$: Add v to $a[i]$ and recompute $b[]$.
- $\text{SUM}(\ell, r)$: Return $b[r] - b[\ell - 1]$.
- Complexity: $\text{ADD}(i, v) \in \mathcal{O}(n)$, $\text{SUM}(\ell, r) \in \mathcal{O}(1)$

Conclusion:

- Use Alg 1 if $\#\text{ADD-Queries} \gg \#\text{SUM-Queries}$.
- Use Alg 2 if $\#\text{ADD-Queries} \ll \#\text{SUM-Queries}$.

Segment Tree - Motivation

Alg 2:

- Compute prefix sums and store them in $b[]$.
- $\text{ADD}(i, v)$: Add v to $a[i]$ and recompute $b[]$.
- $\text{SUM}(\ell, r)$: Return $b[r] - b[\ell - 1]$.
- Complexity: $\text{ADD}(i, v) \in \mathcal{O}(n)$, $\text{SUM}(\ell, r) \in \mathcal{O}(1)$

Conclusion:

- Use Alg 1 if $\#\text{ADD-Queries} \gg \#\text{SUM-Queries}$.
- Use Alg 2 if $\#\text{ADD-Queries} \ll \#\text{SUM-Queries}$.

What if $\#\text{ADD-Queries} \approx \#\text{SUM-Queries}$?

Segment Tree

A *Segment Tree* is a tree data structure used for storing information about (integer) intervals.

Segment Tree

A *Segment Tree* is a tree data structure used for storing information about (integer) intervals.

- A segment tree is a binary tree.

Segment Tree

A *Segment Tree* is a tree data structure used for storing information about (integer) intervals.

- A segment tree is a binary tree.
- Each node stores a value v for an interval $[\ell, r]$.

Segment Tree

A *Segment Tree* is a tree data structure used for storing information about (integer) intervals.

- A segment tree is a binary tree.
- Each node stores a value v for an interval $[\ell, r]$.
- Root represents the full interval $[0, n - 1]$.

Segment Tree

A *Segment Tree* is a tree data structure used for storing information about (integer) intervals.

- A segment tree is a binary tree.
- Each node stores a value v for an interval $[\ell, r]$.
- Root represents the full interval $[0, n - 1]$.
- If node v represents interval $[\ell, r]$, its left child represents $[\ell, m]$ and its right child $[m + 1, r]$ where $m = \lfloor (\ell + r)/2 \rfloor$.

Segment Tree

A *Segment Tree* is a tree data structure used for storing information about (integer) intervals.

- A segment tree is a binary tree.
- Each node stores a value v for an interval $[\ell, r]$.
- Root represents the full interval $[0, n - 1]$.
- If node v represents interval $[\ell, r]$, its left child represents $[\ell, m]$ and its right child $[m + 1, r]$ where $m = \lfloor (\ell + r)/2 \rfloor$.
- Leaves represent unit-intervals $[t, t]$.

Segment Tree

A *Segment Tree* is a tree data structure used for storing information about (integer) intervals.

- A segment tree is a binary tree.
 - Each node stores a value v for an interval $[\ell, r]$.
 - Root represents the full interval $[0, n - 1]$.
 - If node v represents interval $[\ell, r]$, its left child represents $[\ell, m]$ and its right child $[m + 1, r]$ where $m = \lfloor (\ell + r)/2 \rfloor$.
 - Leaves represent unit-intervals $[t, t]$.
-
- Segment trees were invented by Jon Louis Bentley in 1977.¹

Segment Tree

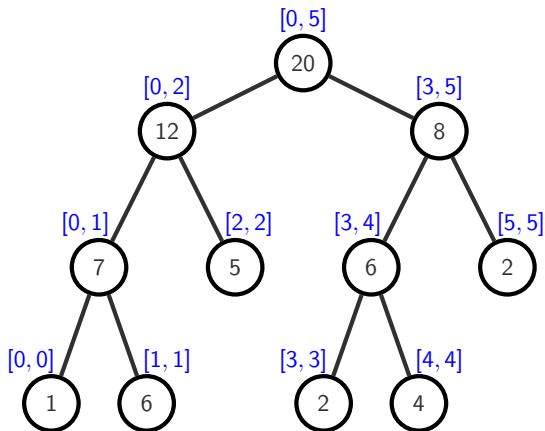
A *Segment Tree* is a tree data structure used for storing information about (integer) intervals.

- A segment tree is a binary tree.
 - Each node stores a value v for an interval $[\ell, r]$.
 - Root represents the full interval $[0, n - 1]$.
 - If node v represents interval $[\ell, r]$, its left child represents $[\ell, m]$ and its right child $[m + 1, r]$ where $m = \lfloor (\ell + r)/2 \rfloor$.
 - Leaves represent unit-intervals $[t, t]$.
-
- Segment trees were invented by Jon Louis Bentley in 1977.¹

¹At least colleagues told me - I could not find the original source.

Segment Tree - Example

Input array: $a[] = [1, 6, 5, 2, 4, 2]$



Segment Tree - Implementation

- Each node holds a value and the associated interval.

```
struct Node {int v,l,r;}
```

Segment Tree - Implementation

- Each node holds a value and the associated interval.

```
struct Node {int v,l,r;}
```

- Represent segment tree as an array of nodes.

Segment Tree - Implementation

- Each node holds a value and the associated interval.

```
struct Node {int v,l,r;}
```

- Represent segment tree as an array of nodes.
- Root node is stored at index zero.

Segment Tree - Implementation

- Each node holds a value and the associated interval.

```
struct Node {int v,l,r;}
```

- Represent segment tree as an array of nodes.
- Root node is stored at index zero.
- In order to refer to children, there are two approaches (common for binary trees):

Segment Tree - Implementation

- Each node holds a value and the associated interval.

```
struct Node {int v,l,r;}
```

- Represent segment tree as an array of nodes.
- Root node is stored at index zero.
- In order to refer to children, there are two approaches (common for binary trees):
 - explicitly store indices of children in node or

Segment Tree - Implementation

- Each node holds a value and the associated interval.

```
struct Node {int v,l,r;}
```

- Represent segment tree as an array of nodes.
- Root node is stored at index zero.
- In order to refer to children, there are two approaches (common for binary trees):
 - explicitly store indices of children in node or
 - fill array with 0s to have size 2^h with h minimal.

Segment Tree - Implementation

- Each node holds a value and the associated interval.

```
struct Node {int v,l,r;}
```

- Represent segment tree as an array of nodes.
- Root node is stored at index zero.
- In order to refer to children, there are two approaches (common for binary trees):
 - explicitly store indices of children in node or
 - fill array with 0s to have size 2^h with h minimal.
- For the latter, the following holds:

Segment Tree - Implementation

- Each node holds a value and the associated interval.

```
struct Node {int v,l,r;}
```

- Represent segment tree as an array of nodes.
- Root node is stored at index zero.
- In order to refer to children, there are two approaches (common for binary trees):
 - explicitly store indices of children in node or
 - fill array with 0s to have size 2^h with h minimal.
- For the latter, the following holds:
 - $h = \lceil \log_2 n \rceil$, where n is the size of the input array

Segment Tree - Implementation

- Each node holds a value and the associated interval.

```
struct Node {int v,l,r;}
```

- Represent segment tree as an array of nodes.
- Root node is stored at index zero.
- In order to refer to children, there are two approaches (common for binary trees):
 - explicitly store indices of children in node or
 - fill array with 0s to have size 2^h with h minimal.
- For the latter, the following holds:
 - $h = \lceil \log_2 n \rceil$, where n is the size of the input array
 - the children of node i are stored at indices $2i + 1$ and $2i + 2$.
(exercise: why?)

Segment Tree - Implementation

- Each node holds a value and the associated interval.

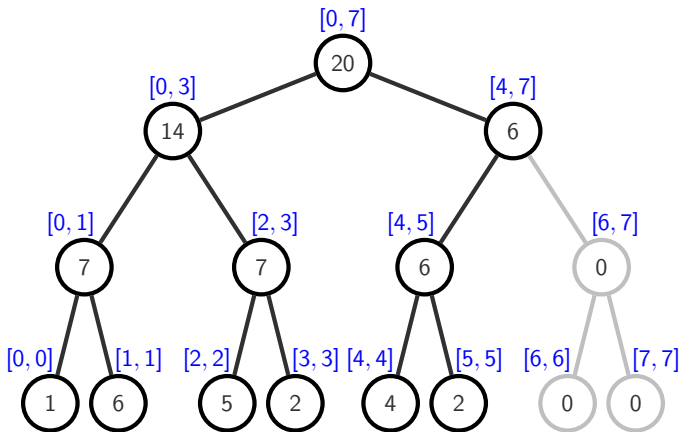
```
struct Node {int v,l,r;}
```

- Represent segment tree as an array of nodes.
- Root node is stored at index zero.
- In order to refer to children, there are two approaches (common for binary trees):
 - explicitly store indices of children in node or
 - fill array with 0s to have size 2^h with h minimal.
- For the latter, the following holds:
 - $h = \lceil \log_2 n \rceil$, where n is the size of the input array
 - the children of node i are stored at indices $2i + 1$ and $2i + 2$.
(exercise: why?)
 - the required size of the node array is $2^{h+1} - 1$

Segment Tree - Example

Input array: $a[] = [1, 6, 5, 2, 4, 2, 0, 0]$

Segment tree values: $v[] = [20, 14, 6, 7, 7, 6, 0, 1, 6, 5, 2, 4, 2, 0, 0]$



Segment Tree - Build

Algorithm Segment Tree - Init

Input: input array $a[]$ of size n

Output: Segment tree rooted at 0 on interval $[0, n - 1]$.

procedure INIT($a[]$)

$pwr \leftarrow 1$

while $pwr < n$ **do**

$pwr \leftarrow pwr \cdot 2$

$t \leftarrow \text{Node}[2 * pwr - 1]$

 BUILD($a, t, 0, 0, pwr - 1$)

return t

Segment Tree - Build

Algorithm Segment Tree - Build

Input: input $a[]$, segment tree $t[]$, current index p , interval $[\ell, r]$.

Output: Builds segment tree rooted at p on interval $[\ell, r]$ and outputs value at p .

procedure BUILD($a[], t[], p, \ell, r$)

$t[p].\ell \leftarrow \ell$

$t[p].r \leftarrow r$

if $\ell = r$ **then**

if $\ell < n$ **then**

$t[p].v \leftarrow a[\ell]$

else

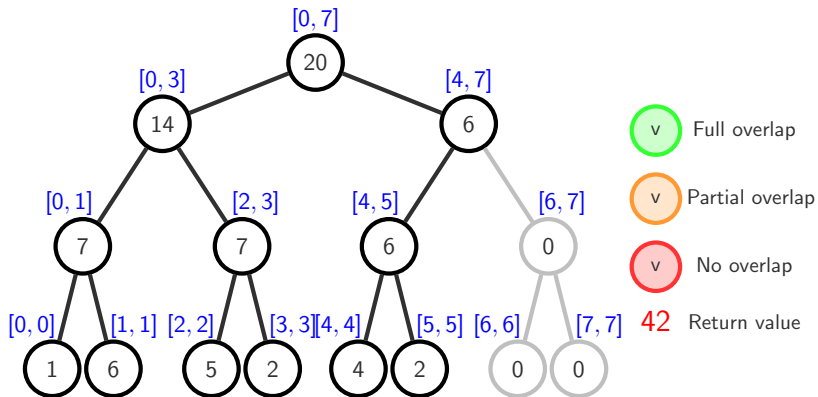
$m \leftarrow \lfloor (\ell + r) / 2 \rfloor$

$t[p].v \leftarrow \text{BUILD}(a, t, 2p + 1, \ell, m) + \text{BUILD}(a, t, 2p + 2, m + 1, r)$

return $t[p].v$

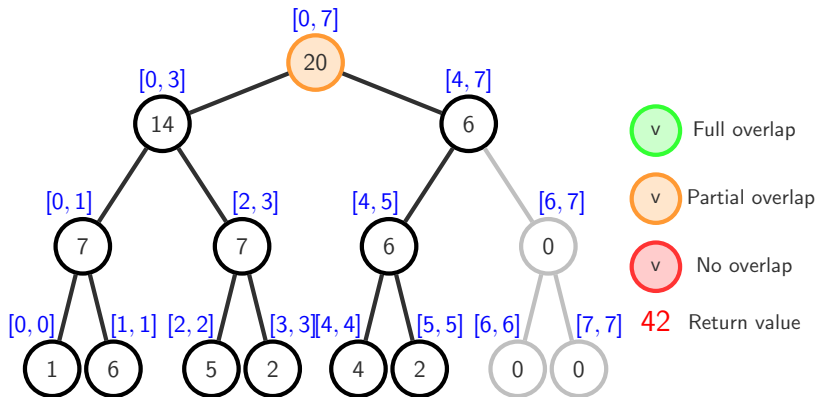
Segment Tree - Sum

SUM(1,4)



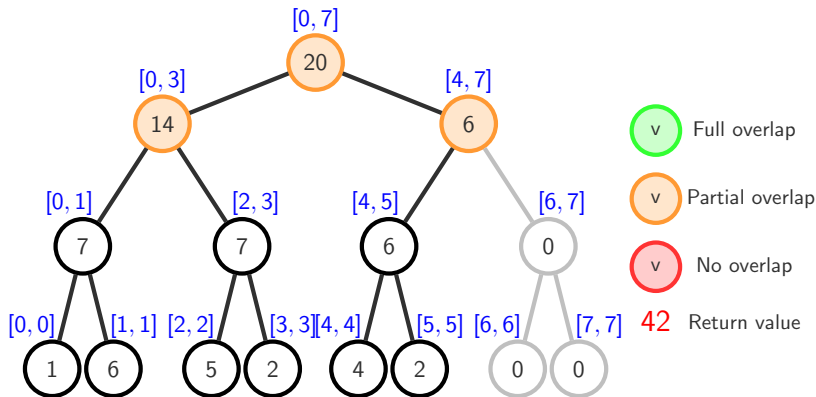
Segment Tree - Sum

SUM(1,4)



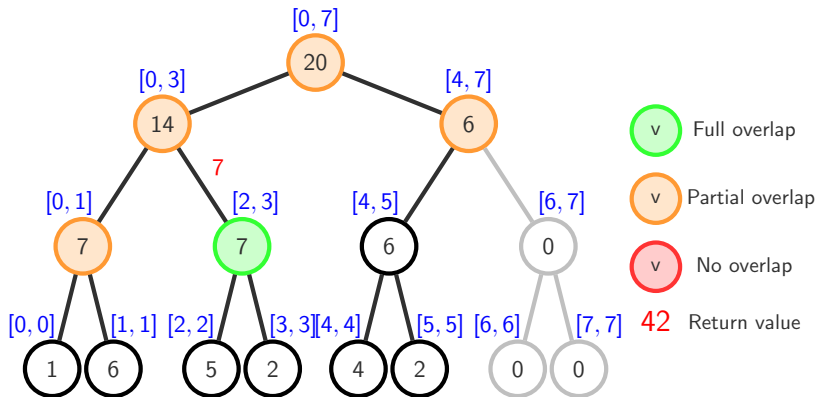
Segment Tree - Sum

SUM(1,4)



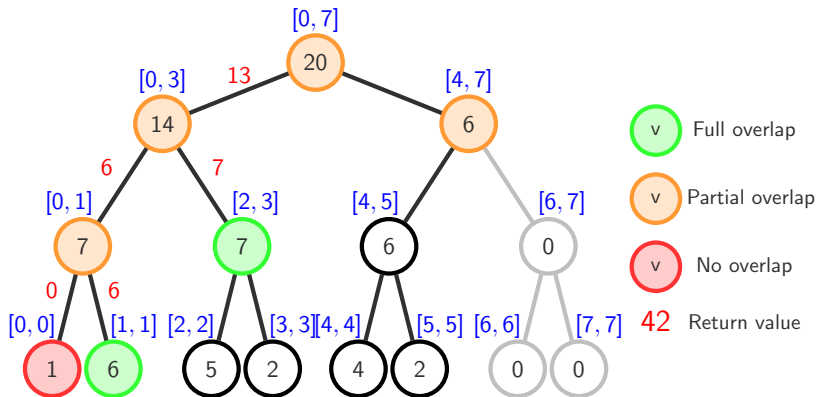
Segment Tree - Sum

SUM(1,4)



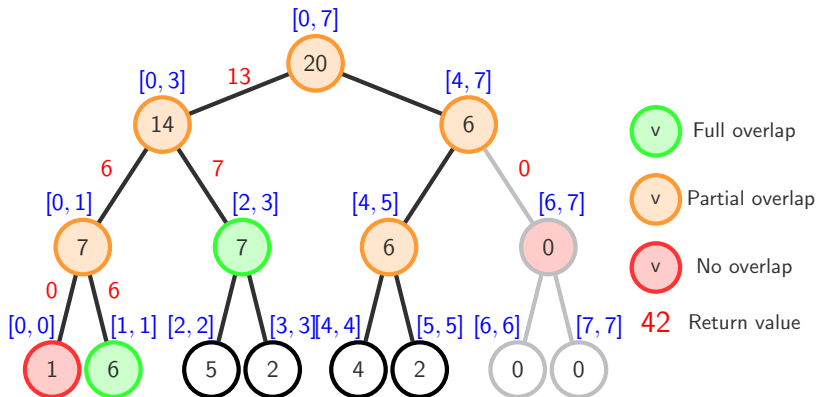
Segment Tree - Sum

SUM(1,4)



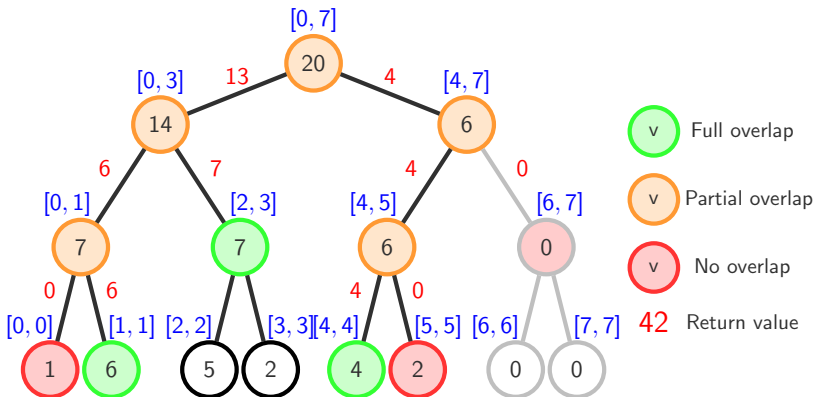
Segment Tree - Sum

SUM(1,4)



Segment Tree - Sum

$$\text{SUM}(1,4) = 17$$



Segment Tree - Sum

Algorithm Segment Tree - Sum

Input: Segment tree $t[]$, current index p , interval $[\ell, r]$.

Output: Sum on interval $[\ell, r]$.

procedure SUM($t[], p, \ell, r$)

if $\ell > t[p].r$ **or** $r < t[p].\ell$ **then**

 ▷ No overlap

return 0

if $\ell \leq t[p].\ell$ **and** $t[p].r \leq r$ **then**

 ▷ Full overlap

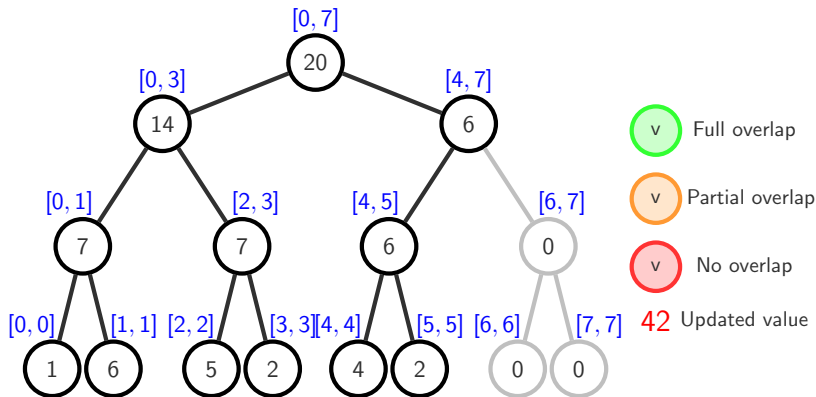
return $t[p].v$

 ▷ Partial overlap

return SUM($t, 2p + 1, \ell, r$) + SUM($t, 2p + 2, \ell, r$)

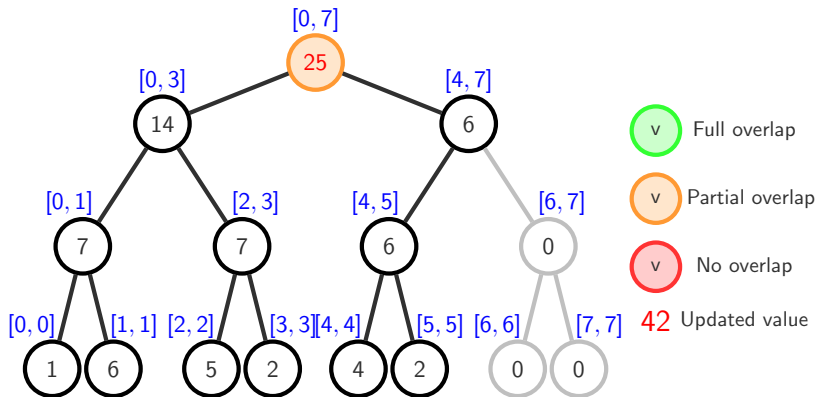
Segment Tree - Add

ADD 5 to index 4



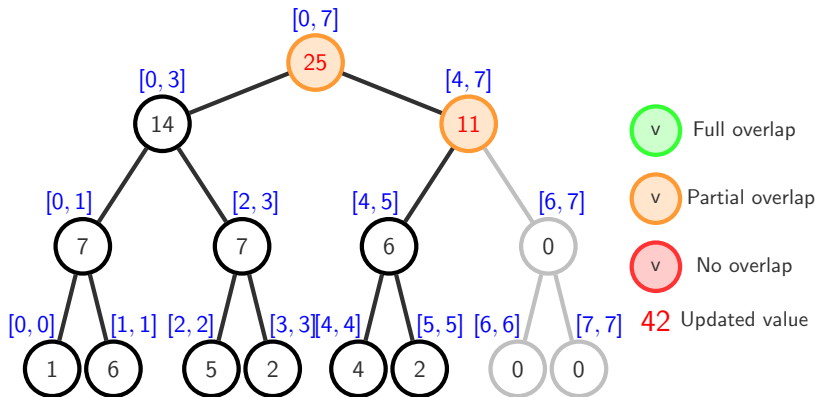
Segment Tree - Add

ADD 5 to index 4



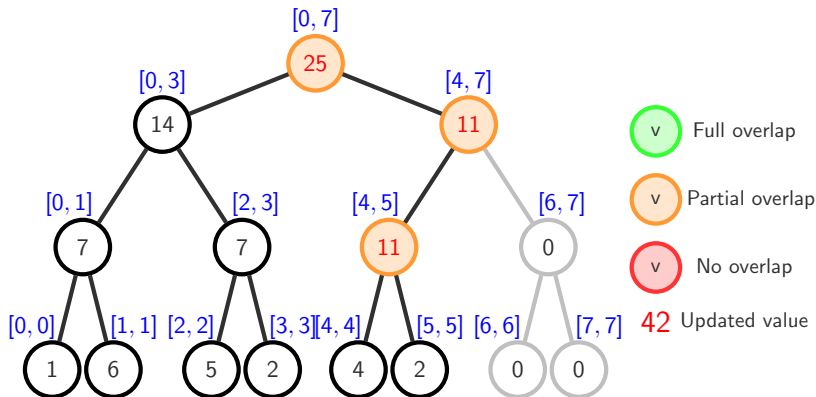
Segment Tree - Add

ADD 5 to index 4



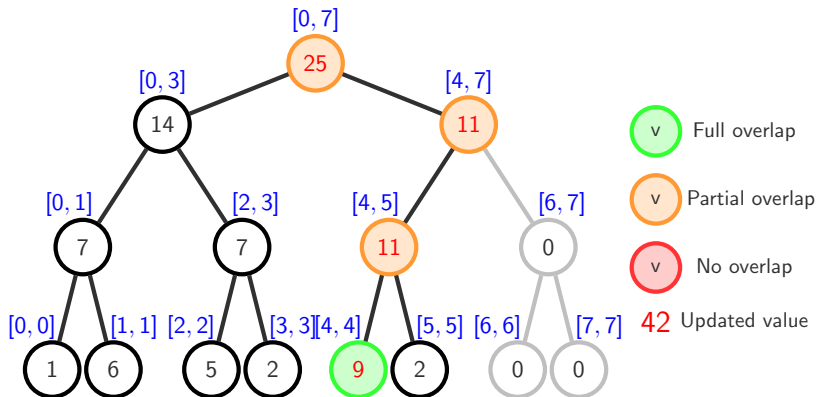
Segment Tree - Add

ADD 5 to index 4



Segment Tree - Add

ADD 5 to index 4



Segment Tree - Add

Algorithm Segment Tree - Add

Input: Segment tree $t[]$, current index p , update index i , update value v .

procedure ADD($t[], p, i, v$)

$t[p].v \leftarrow t[p].v + v$

if $t[p].\ell = t[p].r$ **then**

return

$m \leftarrow \lfloor (t[p].\ell + t[p].r) / 2 \rfloor$

if $i \leq m$ **then**

 ADD($t, 2p + 1, i, v$)

else

 ADD($t, 2p + 2, i, v$)

Segment Tree - Complexity

Complexity

Let n be the size of the input array $a[]$.

- BUILD: $pwr \leq 2n - 2$, and the tree has $2pwr - 1 \leq 4n$ nodes. Each node is visited once. $\mathcal{O}(n)$.
- ADD: Exactly one node visited on every level. $\mathcal{O}(\log n)$.
- SUM: At most four nodes are visited on every level. $\mathcal{O}(\log n)$.

Algebra (why is he showing this?)

Definition

Let S be a set, and $\bullet : S \times S \rightarrow S$.

Algebra (why is he showing this?)

Definition

Let S be a set, and $\bullet : S \times S \rightarrow S$.

- S is called *semigroup* if $\bullet$ is associative, i.e. if for all $x, y, z \in S$ it holds that $(x \bullet y) \bullet z = x \bullet (y \bullet z)$.

Algebra (why is he showing this?)

Definition

Let S be a set, and $\bullet : S \times S \rightarrow S$.

- S is called *semigroup* if $\bullet$ is associative, i.e. if for all $x, y, z \in S$ it holds that $(x \bullet y) \bullet z = x \bullet (y \bullet z)$.
- S is called *monoid* if it is a semigroup and there is a neutral element, i.e. some $e \in S$ s.t. for all $x \in S$ it holds that $e \bullet x = x = x \bullet e$.

Algebra (why is he showing this?)

Definition

Let S be a set, and $\bullet : S \times S \rightarrow S$.

- S is called *semigroup* if $\bullet$ is associative, i.e. if for all $x, y, z \in S$ it holds that $(x \bullet y) \bullet z = x \bullet (y \bullet z)$.
- S is called *monoid* if it is a semigroup and there is a neutral element, i.e. some $e \in S$ s.t. for all $x \in S$ it holds that $e \bullet x = x = x \bullet e$.

Remark

Let S be a semigroup without neutral element. Then, S can be made into a monoid $S \uplus \{e\}$ by setting $e \bullet x := x := x \bullet e$ for all $x \in S \uplus \{e\}$.

Monoids - examples

Examples

The following structures $(S, \bullet, e)$ are monoids:

Monoids - examples

Examples

The following structures $(S, \bullet, e)$ are monoids:

- $(\mathbb{N}, +, 0)$, $(\mathbb{Q}, +, 0)$, $(\mathbb{R}, +, 0)$, ...

Monoids - examples

Examples

The following structures $(S, \bullet, e)$ are monoids:

- $(\mathbb{N}, +, 0)$, $(\mathbb{Q}, +, 0)$, $(\mathbb{R}, +, 0)$, ...
- $(\mathbb{R} \uplus \{\infty\}, \min, \infty)$, $(\mathbb{R} \uplus \{-\infty\}, \max, -\infty)$, ...

Monoids - examples

Examples

The following structures $(S, \bullet, e)$ are monoids:

- $(\mathbb{N}, +, 0)$, $(\mathbb{Q}, +, 0)$, $(\mathbb{R}, +, 0)$, ...
- $(\mathbb{R} \uplus \{\infty\}, \min, \infty)$, $(\mathbb{R} \uplus \{-\infty\}, \max, -\infty)$, ...
- $(2^{\mathbb{N}}, \triangle, \emptyset)$ where $A \triangle B := (A \setminus B) \cup (B \setminus A)$

Monoids - examples

Examples

The following structures $(S, \bullet, e)$ are monoids:

- $(\mathbb{N}, +, 0)$, $(\mathbb{Q}, +, 0)$, $(\mathbb{R}, +, 0)$, ...
- $(\mathbb{R} \uplus \{\infty\}, \min, \infty)$, $(\mathbb{R} \uplus \{-\infty\}, \max, -\infty)$, ...
- $(2^{\mathbb{N}}, \triangle, \emptyset)$ where $A \triangle B := (A \setminus B) \cup (B \setminus A)$
- $(\mathbb{R}^{n \times n}, \cdot, I_n)$ (matrices with matrix multiplication)

Monoids - examples

Examples

The following structures $(S, \bullet, e)$ are monoids:

- $(\mathbb{N}, +, 0)$, $(\mathbb{Q}, +, 0)$, $(\mathbb{R}, +, 0)$, ...
- $(\mathbb{R} \uplus \{\infty\}, \min, \infty)$, $(\mathbb{R} \uplus \{-\infty\}, \max, -\infty)$, ...
- $(2^{\mathbb{N}}, \triangle, \emptyset)$ where $A \triangle B := (A \setminus B) \cup (B \setminus A)$
- $(\mathbb{R}^{n \times n}, \cdot, I_n)$ (matrices with matrix multiplication)
- $(\Sigma^*, \cdot, \varepsilon)$ (strings/lists with concatenation)

Monoids - examples

Examples

The following structures $(S, \bullet, e)$ are monoids:

- $(\mathbb{N}, +, 0)$, $(\mathbb{Q}, +, 0)$, $(\mathbb{R}, +, 0)$, ...
- $(\mathbb{R} \uplus \{\infty\}, \min, \infty)$, $(\mathbb{R} \uplus \{-\infty\}, \max, -\infty)$, ...
- $(2^{\mathbb{N}}, \triangle, \emptyset)$ where $A \triangle B := (A \setminus B) \cup (B \setminus A)$
- $(\mathbb{R}^{n \times n}, \cdot, I_n)$ (matrices with matrix multiplication)
- $(\Sigma^*, \cdot, \varepsilon)$ (strings/lists with concatenation)

Segment trees do not only work for sums but for all semigroups!

Monoids - examples

Examples

The following structures $(S, \bullet, e)$ are monoids:

- $(\mathbb{N}, +, 0)$, $(\mathbb{Q}, +, 0)$, $(\mathbb{R}, +, 0)$, ...
- $(\mathbb{R} \uplus \{\infty\}, \min, \infty)$, $(\mathbb{R} \uplus \{-\infty\}, \max, -\infty)$, ...
- $(2^{\mathbb{N}}, \triangle, \emptyset)$ where $A \triangle B := (A \setminus B) \cup (B \setminus A)$
- $(\mathbb{R}^{n \times n}, \cdot, I_n)$ (matrices with matrix multiplication)
- $(\Sigma^*, \cdot, \varepsilon)$ (strings/lists with concatenation)

Segment trees do not only work for sums but for all semigroups!

- Neutral element not strictly required, but makes life easier (can easily fill up array with neutral element and not have to worry any further)

Monoids - examples

Examples

The following structures $(S, \bullet, e)$ are monoids:

- $(\mathbb{N}, +, 0)$, $(\mathbb{Q}, +, 0)$, $(\mathbb{R}, +, 0)$, ...
- $(\mathbb{R} \uplus \{\infty\}, \min, \infty)$, $(\mathbb{R} \uplus \{-\infty\}, \max, -\infty)$, ...
- $(2^{\mathbb{N}}, \triangle, \emptyset)$ where $A \triangle B := (A \setminus B) \cup (B \setminus A)$
- $(\mathbb{R}^{n \times n}, \cdot, I_n)$ (matrices with matrix multiplication)
- $(\Sigma^*, \cdot, \varepsilon)$ (strings/lists with concatenation)

Segment trees do not only work for sums but for all semigroups!

- Neutral element not strictly required, but makes life easier (can easily fill up array with neutral element and not have to worry any further)
- Caution: for non-abelian monoids (e.g. matrices with multiplication, strings with concatenation), need to take care of order of arguments!

Segment Tree - RangeAdd

Implement a new operation:

- $\text{ADD}(i, v)$: Add value v to i -th element.
- $\text{SUM}(\ell, r)$: Sum up all elements in interval $[\ell, r]$.
- $\text{RANGEADD}(\ell, r, v)$: Add value v to each element in range $[\ell, r]$.

Segment Tree - RangeAdd

Implement a new operation:

- $\text{ADD}(i, v)$: Add value v to i -th element.
- $\text{SUM}(\ell, r)$: Sum up all elements in interval $[\ell, r]$.
- $\text{RANGEADD}(\ell, r, v)$: Add value v to each element in range $[\ell, r]$.

Approaches:

Segment Tree - RangeAdd

Implement a new operation:

- $\text{ADD}(i, v)$: Add value v to i -th element.
- $\text{SUM}(\ell, r)$: Sum up all elements in interval $[\ell, r]$.
- $\text{RANGEADD}(\ell, r, v)$: Add value v to each element in range $[\ell, r]$.

Approaches:

- Naive: call $\text{ADD}(i, v)$ for all $i \in [\ell, r]$. Complexity: $\mathcal{O}(n \log n)$.

Segment Tree - RangeAdd

Implement a new operation:

- $\text{ADD}(i, v)$: Add value v to i -th element.
- $\text{SUM}(\ell, r)$: Sum up all elements in interval $[\ell, r]$.
- **$\text{RANGEADD}(\ell, r, v)$: Add value v to each element in range $[\ell, r]$.**

Approaches:

- Naive: call $\text{ADD}(i, v)$ for all $i \in [\ell, r]$. Complexity: $\mathcal{O}(n \log n)$.
- Better: update all nodes whose interval intersects $[\ell, r]$. Complexity: $\mathcal{O}(n)$.

Segment Tree - RangeAdd

Implement a new operation:

- $\text{ADD}(i, v)$: Add value v to i -th element.
- $\text{SUM}(\ell, r)$: Sum up all elements in interval $[\ell, r]$.
- **$\text{RANGEADD}(\ell, r, v)$: Add value v to each element in range $[\ell, r]$.**

Approaches:

- Naive: call $\text{ADD}(i, v)$ for all $i \in [\ell, r]$. Complexity: $\mathcal{O}(n \log n)$.
- Better: update all nodes whose interval intersects $[\ell, r]$. Complexity: $\mathcal{O}(n)$.

Looks good! Let's move on :)

Segment Tree - RangeAdd

Implement a new operation:

- $\text{ADD}(i, v)$: Add value v to i -th element.
- $\text{SUM}(\ell, r)$: Sum up all elements in interval $[\ell, r]$.
- **$\text{RANGEADD}(\ell, r, v)$: Add value v to each element in range $[\ell, r]$.**

Approaches:

- Naive: call $\text{ADD}(i, v)$ for all $i \in [\ell, r]$. Complexity: $\mathcal{O}(n \log n)$.
- Better: update all nodes whose interval intersects $[\ell, r]$. Complexity: $\mathcal{O}(n)$.

~~Looks good! Let's move on :)~~

- $\mathcal{O}(\log n)$ possible!

Segment Tree - RangeAdd

Implement a new operation:

- $\text{ADD}(i, v)$: Add value v to i -th element.
- $\text{SUM}(\ell, r)$: Sum up all elements in interval $[\ell, r]$.
- **$\text{RANGEADD}(\ell, r, v)$: Add value v to each element in range $[\ell, r]$.**

Approaches:

- Naive: call $\text{ADD}(i, v)$ for all $i \in [\ell, r]$. Complexity: $\mathcal{O}(n \log n)$.
- Better: update all nodes whose interval intersects $[\ell, r]$. Complexity: $\mathcal{O}(n)$.

~~Looks good! Let's move on :)~~

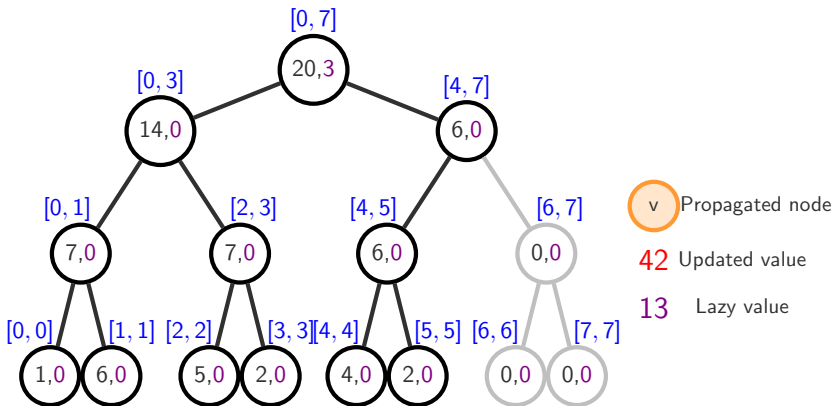
- $\mathcal{O}(\log n)$ possible! Idea: just be lazy...

Segment Tree - Lazy Propagation

- Store an additional integer value *lazy* in each node.
- Do not apply updates immediately but push them to the lazy variable.
- Only propagate lazy value to children when value of node is queried.

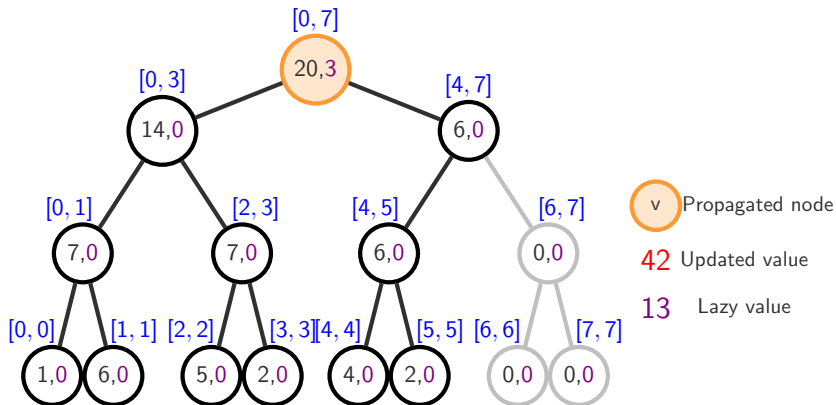
Segment Tree - Propagate

PROPAGATE(0)



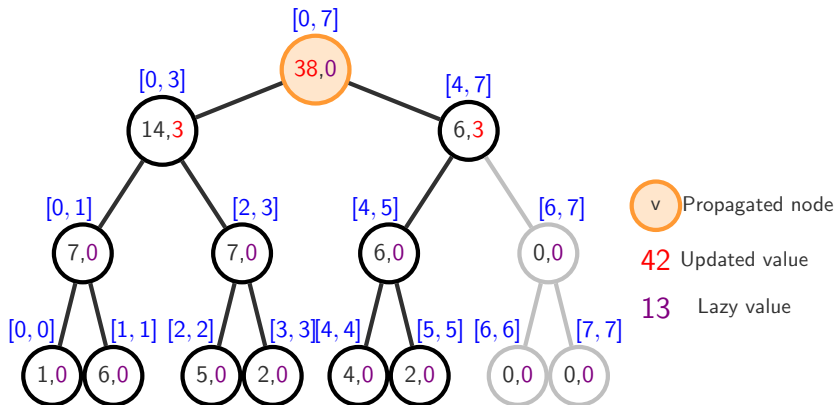
Segment Tree - Propagate

PROPAGATE(0)



Segment Tree - Propagate

PROPAGATE(0)



Segment Tree - Propagate

Algorithm Segment Tree - Propagate

Input: Segment tree $t[]$, current index p .

procedure PROPAGATE($t[], p$)

$t[p].v \leftarrow t[p].v + (t[p].r - t[p].\ell + 1) \cdot t[p].lazy$

if $t[p].\ell \neq t[p].r$ **then**

$t[2p + 1].lazy \leftarrow t[2p + 1].lazy + t[p].lazy$

$t[2p + 2].lazy \leftarrow t[2p + 2].lazy + t[p].lazy$

$t[p].lazy \leftarrow 0$

Segment Tree - Propagate

Algorithm Segment Tree - Propagate

Input: Segment tree $t[]$, current index p .

procedure PROPAGATE($t[], p$)

$t[p].v \leftarrow t[p].v + (t[p].r - t[p].\ell + 1) \cdot t[p].lazy$

if $t[p].\ell \neq t[p].r$ **then**

$t[2p + 1].lazy \leftarrow t[2p + 1].lazy + t[p].lazy$

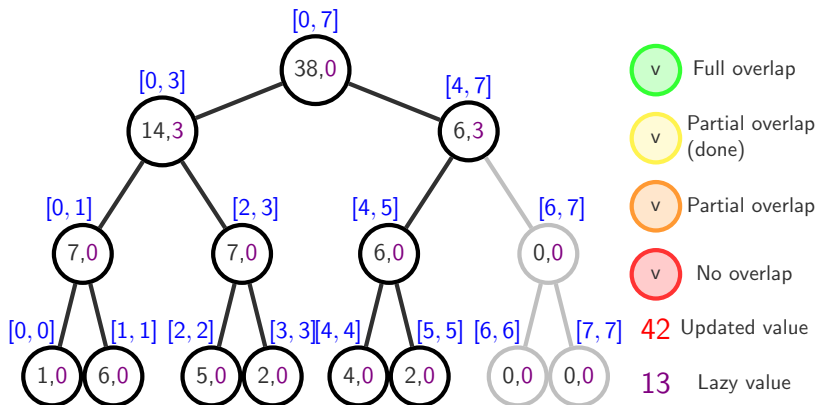
$t[2p + 2].lazy \leftarrow t[2p + 2].lazy + t[p].lazy$

$t[p].lazy \leftarrow 0$

PROPAGATE runs in $\mathcal{O}(1)$!

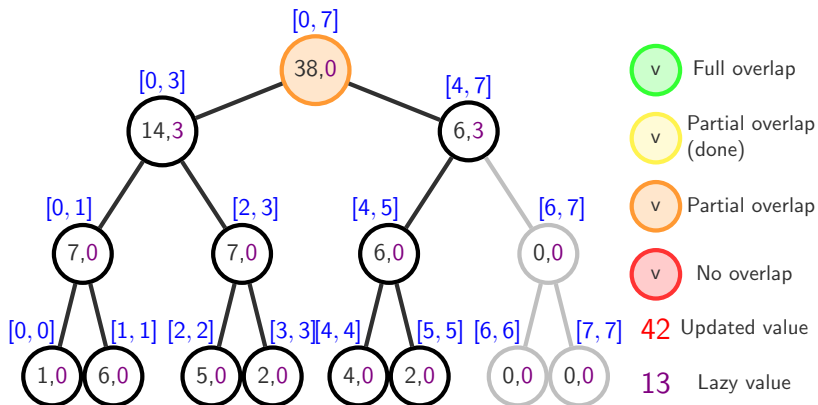
Segment Tree - RangeAdd

RANGEADD 2 on interval $[0, 4]$



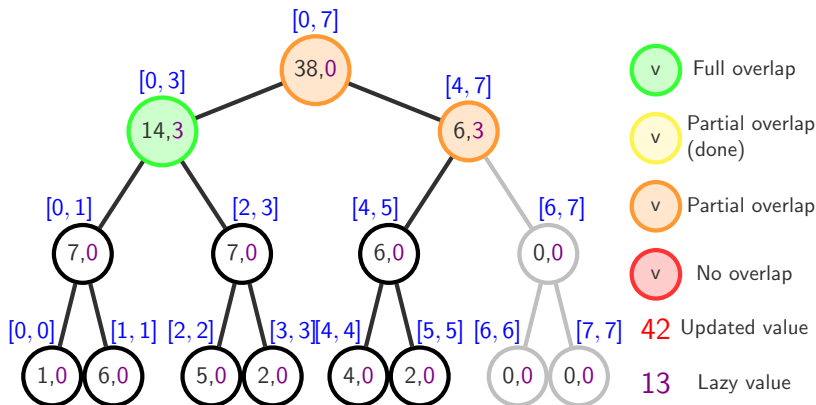
Segment Tree - RangeAdd

RANGEADD 2 on interval $[0, 4]$



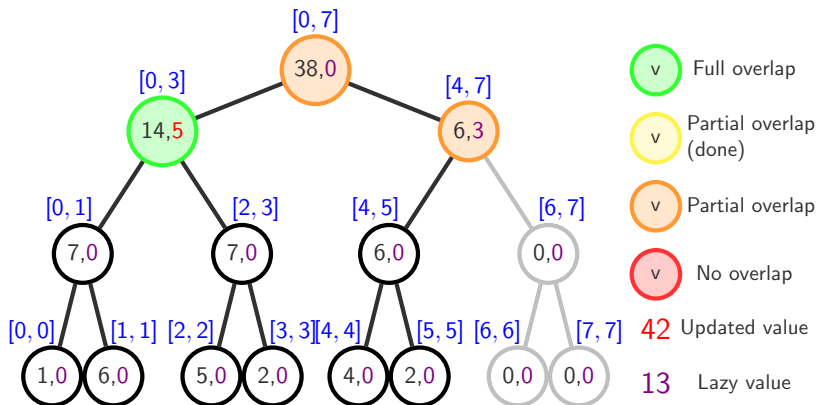
Segment Tree - RangeAdd

RANGEADD 2 on interval $[0, 4]$



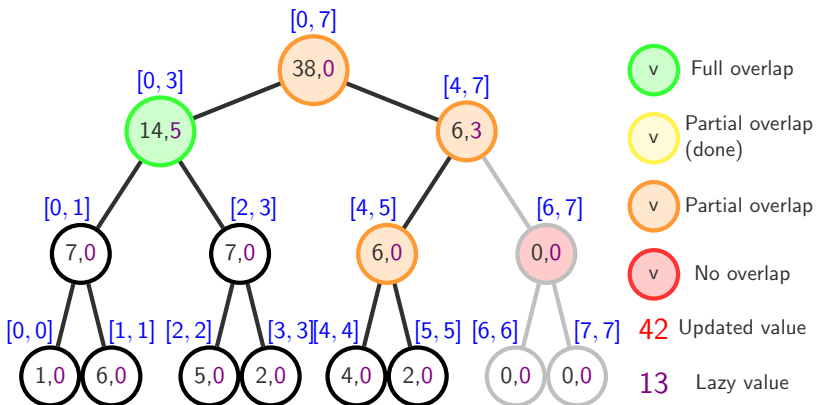
Segment Tree - RangeAdd

RANGEADD 2 on interval $[0, 4]$



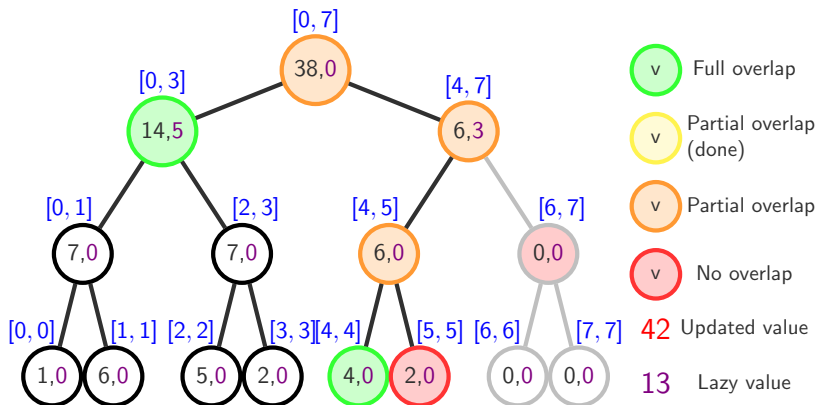
Segment Tree - RangeAdd

RANGEADD 2 on interval $[0, 4]$



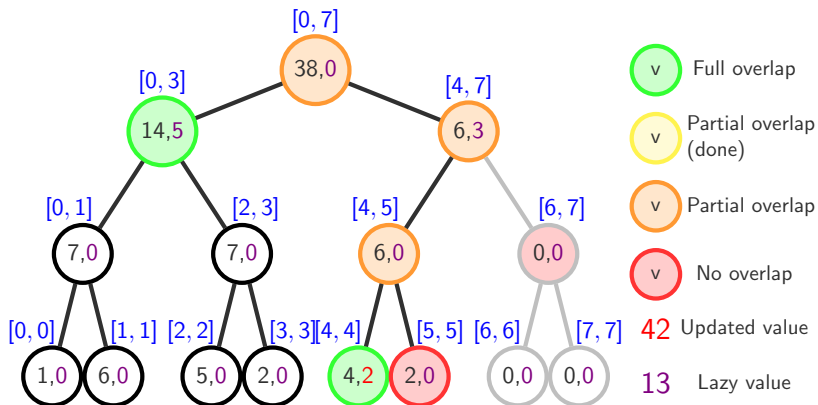
Segment Tree - RangeAdd

RANGEADD 2 on interval $[0, 4]$



Segment Tree - RangeAdd

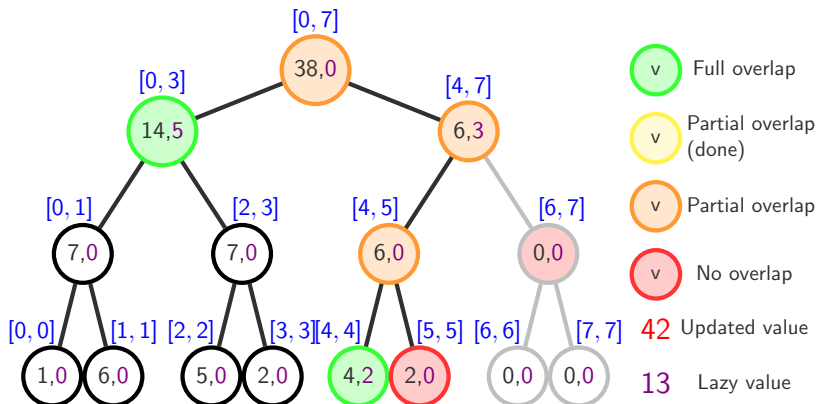
RANGEADD 2 on interval $[0, 4]$



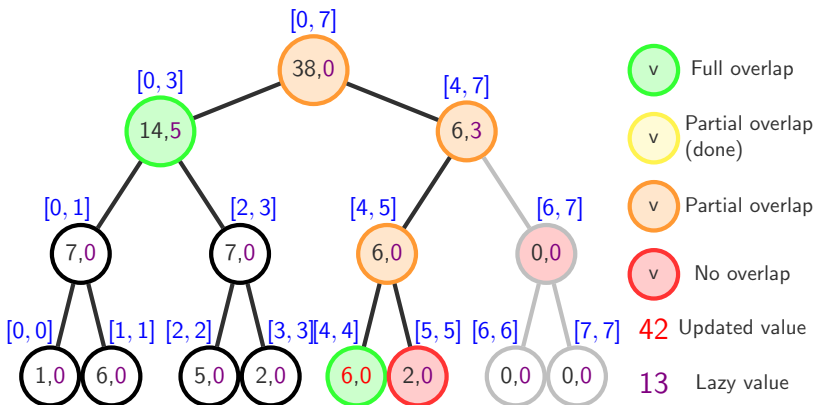
Segment Tree - RangeAdd

RANGEADD 2 on interval $[0, 4]$

node $[4, 5]$ needs to update $\rightarrow$ need values of children $\rightarrow$ propagate

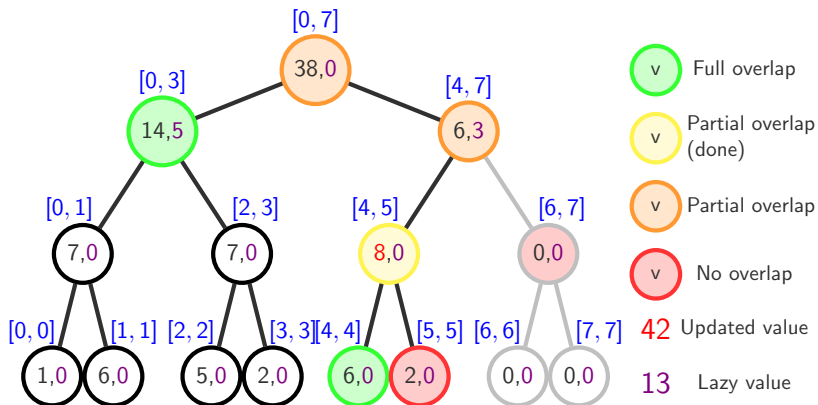


node [4,5] needs to update $\rightarrow$ need values of children $\rightarrow$ propagate



Segment Tree - RangeAdd

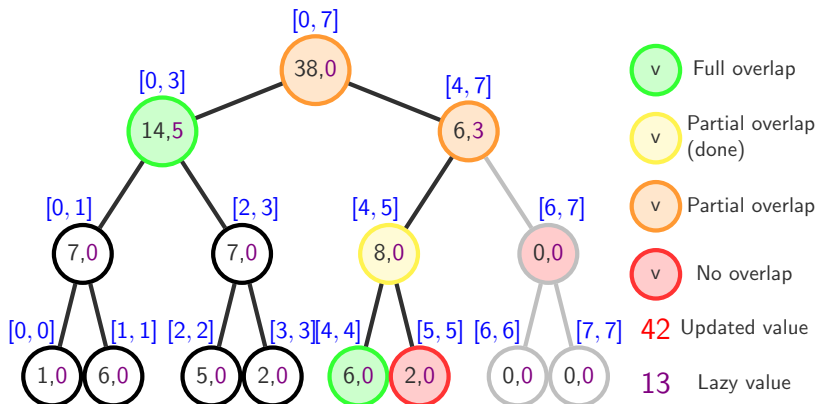
RANGEADD 2 on interval $[0, 4]$



Segment Tree - RangeAdd

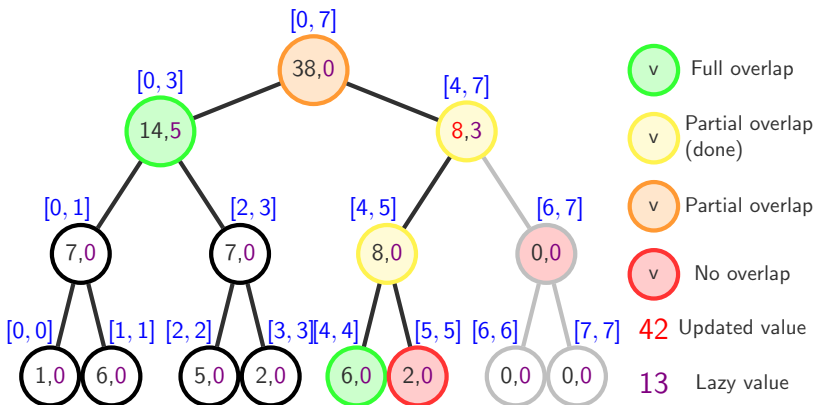
RANGEADD 2 on interval $[0, 4]$

node $[4, 7]$ needs to update $\rightarrow$ need values of children $\rightarrow$ propagate



Segment Tree - RangeAdd

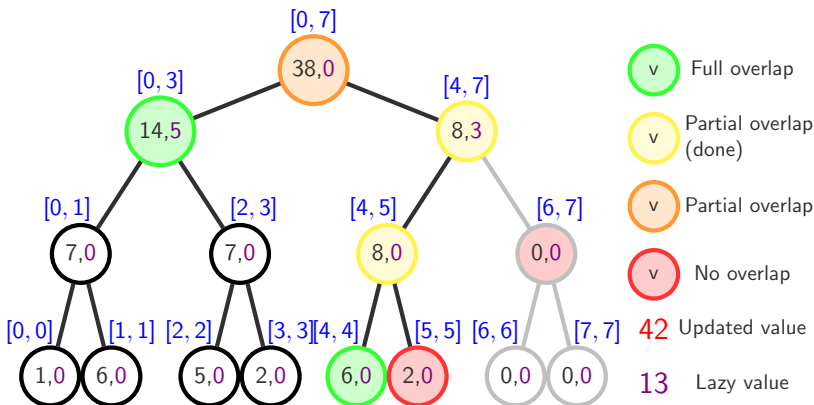
RANGEADD 2 on interval $[0, 4]$



Segment Tree - RangeAdd

RANGEADD 2 on interval $[0, 4]$

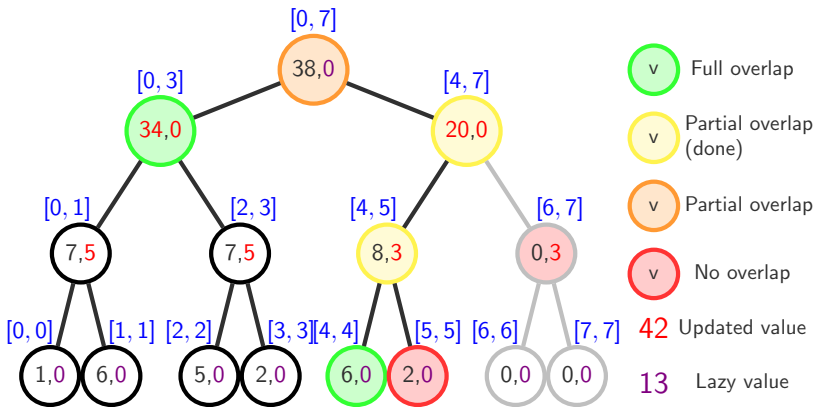
node $[0, 7]$ needs to update $\rightarrow$ need values of children $\rightarrow$ propagate



Segment Tree - RangeAdd

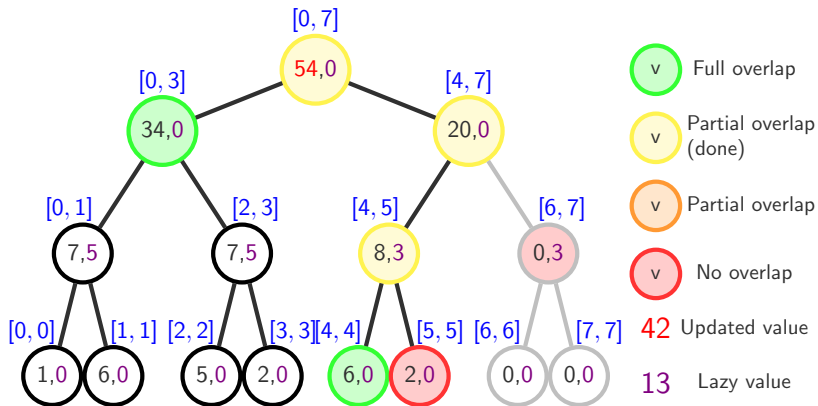
RANGEADD 2 on interval $[0, 4]$

node $[0, 7]$ needs to update $\rightarrow$ need values of children $\rightarrow$ propagate



Segment Tree - RangeAdd

RANGEADD 2 on interval $[0, 4]$



Segment Tree - RangeAdd

Algorithm Segment Tree - RangeAdd

Input: Segment tree $t[]$, current index p , interval $[\ell, r]$, value v .

```
procedure RANGEADD( $t[], p, \ell, r, v$ )  
    if  $\ell > t[p].r$  or  $r < t[p].\ell$  then  
        return  
    if  $\ell \leq t[p].\ell$  and  $t[p].r \leq r$  then  
         $t[p].lazy \leftarrow t[p].lazy + v$   
    else if  $t[p].\ell \neq t[p].r$  then  
        RANGEADD( $t, 2p + 1, \ell, r, v$ )  
        RANGEADD( $t, 2p + 2, \ell, r, v$ )  
        PROPAGATE( $t, 2p + 1$ )  
        PROPAGATE( $t, 2p + 2$ )  
         $t[p].v \leftarrow t[2p + 1].v + t[2p + 2].v$ 
```

Segment Tree - Sum

Algorithm Segment Tree - Sum

Input: Segment tree $t[]$, current index p , interval $[\ell, r]$.

Output: Sum on interval $[\ell, r]$.

procedure SUM($t[], p, \ell, r$)

if $\ell > t[p].r$ **or** $r < t[p].\ell$ **then**

return 0

PROPAGATE(t, p)

if $\ell \leq t[p].\ell$ **and** $t[p].r \leq r$ **then**

return $t[p].v$

return SUM($t, 2p + 1, \ell, r$) + SUM($t, 2p + 2, \ell, r$)

Segment Tree - Complexity

Complexity

Let n be the size of the input array $a[]$.

- BUILD: $\mathcal{O}(n)$.
- ADD: Exactly one node visited on every level. $\mathcal{O}(\log n)$.
- SUM: At most four nodes are visited on every level. $\mathcal{O}(\log n)$.
- RANGEADD: At most four nodes are visited on every level. $\mathcal{O}(\log n)$.